

Student Support Service Chatbot

Project Report

1. Introduction

The **Student Support Service Chatbot** is an AI-powered conversational assistant designed to answer common student queries. It provides instant responses related to college services, registration, support hours, and general student questions. The project demonstrates a lightweight, hybrid chatbot architecture that combines rule-based pattern matching, a small machine-learning classifier, and optional generative AI fallback through the Gemini API.

2. Objectives

- Provide 24/7 automated support for frequently asked student questions.
- Reduce the workload on human support staff by handling routine queries.
- Deliver fast and consistent responses through pattern matching and ML classification.
- Offer a simple, responsive web interface for students to interact with the chatbot.
- Demonstrate a clean fallback strategy when neither rules nor the ML model can answer.

3. Scope of the Project

- **Web Chat Interface:** A browser-based UI where users type questions and receive replies.
- **Backend API:** A Flask server that processes messages and returns chatbot responses as JSON.
- **Hybrid Response Engine:** Rule-based patterns, an ML pipeline, and an optional Gemini API fallback.
- **Model Training:** A small scikit-learn pipeline trained on example student queries and saved using joblib.
- **Automated Tests:** Unit tests covering greetings, hours, registration, and unknown questions.

4. Technology Stack

Layer	Technology	Purpose
-------	------------	---------

Frontend	HTML5, CSS3, JavaScript	Chat UI, message rendering, API calls
Backend	Python, Flask	REST API for chat messages
Machine Learning	scikit-learn, joblib	TF-IDF + Logistic Regression classifier
External AI	Google Gemini API (optional)	Generative fallback for unknown queries
HTTP	requests	Calling the Gemini REST endpoint
Testing	unittest	Unit tests for core chatbot logic
Version Control	Git, GitHub	Source code management

5. Project Structure

File	Description
<code>app.py</code>	Flask application; serves the static frontend and exposes <code>/api/chat</code> .
<code>student_support_chatbot.py</code>	Core chatbot class containing pattern rules, ML inference, and Gemini fallback.
<code>train_model.py</code>	Trains a TF-IDF + Logistic Regression pipeline and saves model artifacts to <code>model/</code> .
<code>index.html</code>	Responsive chat interface.
<code>styles.css</code>	UI styling for the chat page.

<code>app.js</code>	Client-side script that handles message submission and displays the conversation.
<code>tests/test_student_support_chatbot.py</code>	Unit tests for chatbot responses.
<code>requirements.txt</code>	Python dependencies: Flask, scikit-learn, joblib, requests.

6. System Architecture

The system follows a simple client-server pattern. The frontend runs in a web browser and sends user messages to the Flask backend via a POST request to `/api/chat`. The backend invokes the `StudentSupportChatbot.respond()` method, which processes the input through three layers:

- ML Classifier:** If a trained `pipeline.joblib` and `responses.joblib` are present, the model predicts a label and returns the mapped response.
- Rule-Based Patterns:** If the ML model is unavailable or the label is not recognized, the chatbot matches the message against regular-expression rules for greetings, support hours, and registration.
- Generative Fallback:** If no rule matches and a `GEMINI_API_KEY` environment variable is set, the chatbot calls the Google Gemini API. Otherwise, it returns a default fallback message.

7. Core Modules

7.1 Chatbot Engine

The `StudentSupportChatbot` class loads optional model artifacts at initialization. Its `respond()` method prioritizes the ML pipeline, then rules, then the generative API. This layered design keeps the chatbot functional even when the model or API is not configured.

7.2 Training Pipeline

`train_model.py` builds a scikit-learn pipeline that vectorizes text with `TfidfVectorizer` (n-grams up to 2) and classifies with `LogisticRegression`. Labels include greeting, hours, registration, and unknown. The trained pipeline and a label-to-response dictionary are persisted in the `model/` directory using `joblib`.

7.3 Web Application

The Flask app serves `index.html` at the root and exposes a single JSON endpoint at `/api/chat`. The endpoint validates the incoming message and returns the chatbot reply or an error response if processing fails.

7.4 Frontend Interface

The interface consists of a chat card with a message display area and a composer form. When a user submits a message, JavaScript appends the message, shows a typing indicator, calls the backend API, and displays the bot reply.

8. Features

- Hybrid response engine: ML + rules + generative AI.
- Lightweight and dependency-light Flask backend.
- Single-page web chat UI with responsive design.
- Command-line chat mode for quick local testing.
- Unit tests for core response behaviors.
- Secure handling of API keys via environment variables.
- Offline fallback when no API key is configured.

9. Use Cases

- A student types "hello" and receives a greeting message.
- A student asks "What are your hours?" and learns the support desk schedule.
- A student asks about class registration and is directed to the registration portal.
- For unrecognized questions, the chatbot either uses Gemini (if configured) or suggests contacting support.

10. Testing

Unit tests in `tests/test_student_support_chatbot.py` verify that the chatbot returns the expected responses for greetings, hours, registration, and unknown inputs. Tests can be run with the standard `unittest` runner or any compatible test framework.

11. Deployment and Usage

To run the project locally:

1. Install dependencies: `pip install -r requirements.txt`
2. (Optional) Train the model: `python train_model.py`
3. Start the server: `python app.py`
4. Open `http://localhost:5000` in a browser.

For generative AI fallback, set the `GEMINI_API_KEY` environment variable. A different model may be selected via `GEMINI_MODEL`.

12. Advantages

- Easy to understand, extend, and deploy.
- Does not require an external API for basic, rule-based operation.
- ML model can be retrained with new training data as the FAQ set grows.
- Modular code makes it simple to add new patterns or integration points.

13. Limitations and Future Scope

Limitations:

- The rule set is small and covers only a few topics.
- The ML model requires manually labeled examples and periodic retraining.
- Conversations are not persisted across sessions by default.

Future Scope:

- Expand the rule and training datasets to cover more student services.
- Add conversation history and user session persistence.
- Integrate a database for storing FAQs, user queries, and feedback.
- Support voice input and multi-language responses.
- Deploy to a cloud platform for public access.

14. Conclusion

The Student Support Service Chatbot is a practical demonstration of a modern, hybrid chatbot built with Python and Flask. Its combination of rule-based matching, machine learning, and optional generative AI makes it both easy to run locally and flexible enough for future

enhancements. The project provides a solid foundation for a fully-featured student support assistant.

Project Repository: [dvrajgit/dev-student-support-service-chatbot](#)

Report Version: 1.0